技　术　交　付　文　档

kafka-file-loader

功 能 文 档

19 项功能 • 端到端链路 • 状态机 • 接口规格 • 边界与限制

依据源码与实测报告编写 • 适用于 1.0.0 版本

文档版本　v1.0.0
适用版本　1.0.0（Spring Boot 2.7.18 / JDK 1.8）
文档性质　项目功能说明书（非部署手册）
编制日期　2026-10-26

目　录

文档说明 1

阅读指引 1

一、项目定位 1

1.1 一句话定位 1

1.2 解决的问题 1

1.3 功能边界 2

1.4 技术栈与代码规模 3

1.5 术语表 3

二、总体架构 3

2.1 逻辑分层 3

2.2 组件职责 4

2.3 端到端数据流 5

2.4 单条消息的完整处理时序 5

2.5 部署形态 6

三、功能清单 6

3.1 功能全景 6

3.2 功能域与角色 7

四、功能详述 8

4.1 数据接入域 8

4.2 文件管道域 11

4.3 文件投递域 15

4.4 稳定性保障域 17

4.5 运维与交付域 18

五、文件与数据规格 21

5.1 文件命名 21
5.2 行格式 21
5.3 31 字段字典 21
5.4 取值规则速查 23
5.5 隔离文件格式 23
5.6 编码与时区 24
六、文件状态机与目录 24
6.1 状态流转 24
6.2 五个目录 24
6.3 各环节失败的处理对照 25
七、接口规格 25
7.1 接口总览 25
7.2 POST /api/admin/files/flush 26
7.3 POST /api/admin/files/upload 26
7.4 GET /api/admin/schema 27
7.5 GET /api/admin/files/stats 27
7.6 GET /api/admin/status 28
7.7 Actuator 29
八、非功能特性 29
8.1 性能实测 29
8.2 可靠性设计矩阵 29
8.3 并发模型 30
8.4 资源占用 30
8.5 安全 31
九、配置项 → 功能映射 31

9.1 影响行为的核心配置 31
9.2 配置改动生效方式 32
十、边界条件与已知限制 32
10.1 数据语义 32
10.2 运维接口与进程 33
10.3 多实例部署 33
10.4 调度与时效 33
10.5 投递与外部依赖 34
十一、测试与验收 34
11.1 测试执行 34
11.2 功能验收清单 34
十二、代码结构地图 36
附录 A 日志关键字 37
附录 B 常见问题 37

文档说明

项	内容
文档名称	kafka-file-loader 项目功能文档
文档版本	v1.0.0
对应代码版本	1.0.0（`pom.xml`）
编写日期	2026-10-26
文档定位	描述系统**做什么、怎么做、边界在哪**；部署步骤见 `docs/deploy.md`
依据	全部结论来自源码与实测报告（`src/main/java`、`src/test/java`、`docs/loadtest.md`），非通用模板
读者	需求方、下游数据接入方、运维、二次开发

阅读指引

你是谁	建议读
想快速知道这东西干什么	第一章 + 第三章功能全景表
下游要建表、写装载脚本	第五章（文件与数据规格）+ 7.4 字段接口
运维要巡检、排障、处理积压	第六章状态机 + 第八章非功能 + 第十章限制 + 附录 A
开发要改字段、加投递目标	第四章功能详述 + 第十二章代码地图
验收 / 交付	第十一章验收清单

一、项目定位

1.1 一句话定位

把 Kafka 主题里的 JSON 消息，按固定 31 列渲染成制表符分隔的 CSV 分片文件，落本地 → 备份 → 上传 FTP；不连数据库，最终交付物就是 FTP 上的文件。

1.2 解决的问题

上游业务系统把数据写入 Kafka 主题 `DISTRIBUTE_SQAJXYRXX_TO`，下游数据平台只接受**文件级投递**（FTP 目录 + 定长列顺序），且要求：

诉求	本系统的回答
不丢数据	写盘 `fsync` 成功才提交 offset；上传确认后才删本地；失败进隔离目录不静默丢弃
文件不能半截	全程「临时名 + 原子改名」：本地 `.writing` → 正式名，远端 `.uploading` → 正式名
下游按位置装载不出错	列数恒为 31、顺序与《数据表字段清单》一致；脏数据不进主文件，另存隔离文件
无人值守	定时封盘 + 定时上传 + 定时清备份 + 启动自愈；异常有背压兜底，不把磁盘写满
现场可改	全部配置外置在 `config/application.yml`，改完重启即生效，不用重新打包

1.3 功能边界

做：

- 单集群、单主题的消息消费（一个实例消费一个 topic）
- 消息体 JSON → 31 列定序 CSV 渲染
- 分片滚动、封盘、崩溃恢复
- 本地备份、FTP 投递、远端字节数校验
- 脏数据隔离、失败文件隔离
- 背压限速、优雅停机
- 运维查询/手动触发接口、健康检查、指标、日志

不做（明确排除，避免误期待）：

不做	说明
不连数据库、不建表、不写库	系统内无任何 JDBC/ORM 依赖，数据一条都不落库
不做数据清洗与转换	字段值原样输出，不做类型转换、长度截断、码值翻译、格式标准化
不做去重	语义是「至少一次」，崩溃/重投会产生重复行，去重由下游负责
不做多主题 / 多集群并行	一个实例一个 topic；多主题需多实例或多份配置
不做消息重放工具	隔离文件保留 offset，重取需人工用 `kafka-console-consumer` 等工具
不做接口鉴权	运维接口无认证，靠网络层隔离（见 8.5）
不做配置热更新	配置启动时一次性绑定，改完必须重启

1.4 技术栈与代码规模

项	值
语言 / 运行时	Java 8（`java.version=1.8`）
框架	Spring Boot 2.7.18、Spring Kafka（`@KafkaListener` 批量监听）
关键库	springdoc-openapi-ui 1.6.15、commons-net 3.9.0、Micrometer（actuator）
JSON 解析	Jackson 流式 API（`JsonParser`，不整树加载）
主代码	30 个 Java 文件，约 3,000 行
测试代码	8 个测试类，2,400 余行，56 个用例（50 功能 + 6 压测）
产物	业务 jar 约 228KB + `lib/` 63 个依赖约 41MB，发布包约 39.5MB

1.5 术语表

术语	含义
分片文件（chunk）	一次封盘产出的一个 CSV 文件，最多 5 万行
封盘（seal）	把正在写的临时文件 `flush + fsync + 原子改名` 成正式文件名，之后不再追加
待上传（pending）	已封盘、尚未完成 FTP 投递的文件
隔离区（reject）	消息体无法映射成 31 列时，原文另存的地方，按天一个文件
失败区（failed）	上传重试超限的文件，需人工处理
系统配置名称	文件名前缀，取自 `file.system-name`，未配置回退 `kafka.cluster.name`
背压（backpressure）	积压或磁盘不足时主动停止消费且不提交 offset，让数据留在 Kafka 里
at-least-once	至少一次投递：宁可重复，不丢；重复由下游处理

二、总体架构

2.1 逻辑分层

层	职责	主要类
配置层	绑定 yml、启动校验、暴露配置项	`config/*Properties`、`ClusterConfig`、`KafkaConsumerConfig`
接入层	节点解析、集群快照、消费者装配	`ConfigKafkaNodeRepository`、`ClusterDefinitionProvider`、`RefreshingConsumerFactory`
消费层	批量接消息、控制 ack 时机	`KafkaRecordBatchListener`

层	职责	主要类
配置层	绑定 yml、启动校验、暴露配置项	`config/*Properties`、`ClusterConfig`、`KafkaConsumerConfig`
渲染层	JSON → 31 列一行文本	`RecordRenderer`、`DataSchema`
管道层	分片写入、滚动、封盘、恢复、隔离	`FileStagingManager`、`StagingWriter`、`FileNameAllocator`、`RejectStore`
投递层	备份、FTP 上传、校验、重试	`FileUploadService`、`CommonsNetFtpUploader`
调度层	空闲封盘、定时上传、备份清理	`FilePipelineScheduler`
接口层	运维 REST + Swagger	`AdminController`、`OpenApiConfig`

2.2 组件职责

组件	职责	不负责
`ClusterDefinitionProvider`	启动时把 yml 解析成不可变集群快照；校验主题与节点	不做热更新
`RefreshingConsumerFactory`	每次建连时从 provider 取最新 broker 地址	不做配置监听
`KafkaRecordBatchListener`	收批 → 交管道 → 成功后 ack	不做渲染与落盘细节
`RecordRenderer`	单条消息转一行；不可映射返回 null	不决定落到哪个文件
`FileStagingManager`	按分区路由写入、滚动、封盘、背压判断、恢复	不碰 FTP
`StagingWriter`	单分区文件句柄、行计数、封盘	不知道列含义
`FileNameAllocator`	生成唯一文件名	不移动文件
`RejectStore`	按天写隔离文件并 fsync	不重投
`FileUploadService`	备份 → 上传 → 删除；失败计数与隔离	不解析文件内容
`CommonsNetFtpUploader`	一次 FTP 连接上传一个文件并校验	不重试
`FilePipelineScheduler`	三个定时任务的触发与异常兜底	不做业务判断

2.3 端到端数据流

```
application.yml (kafka.cluster.bootstrap-servers × 16)
        │ 启动解析 + 校验
        ▼
ClusterDefinitionProvider ── 不可变快照 ──► KafkaConsumer（group=kafka-hive-loader）
        ▲
        │ topic: DISTRIBUTE_SQAJXYRXX_TO，500 条/批，3 并发
Kafka ──┘
        │
        ▼
KafkaRecordBatchListener.onBatch()
        │
        ▼
FileStagingManager.appendBatch()
        ├── ① 背压检查（pending ≥ 2000 或磁盘 < 2GB → 抛异常、不 ack）
        ├── ② RecordRenderer 逐条渲染成 31 列（\t 分隔）
        │       └ 非 JSON 对象 → RejectStore 隔离（fsync）→ 该条不进主文件
        ├── ③ 按 (topic, partition) 分组写入 StagingWriter
        │       ├ 满 50,000 行 → 封盘 → 原子改名 → 移入 pending/
        │       └ 每批 flush + fsync（fsync-per-batch=true）
        └── ④ 全部成功才 ack（提交 offset）
        │
        ▼
pending/系统名_yyyyMMddHHmmssSSS.csv
        │ ① 复制到 backup/dt=YYYY-MM-DD/ 并 fsync（失败则不上传）
        │ ② FTP 上传：远端先 .uploading → rename 正式名
        │ ③ 回读远端字节数，与本地一致才算成功
        │ ④ 删本地 pending（备份保留 7 天）
        ▼
FTP /kafka/incoming/  ← 终点，流程结束（不入库）
```

2.4 单条消息的完整处理时序

#	动作	代码位置	失败后果
1	批量拉取 ≤500 条	`KafkaConsumerConfig`（`max-poll-records`）	—
2	进入监听方法	`KafkaRecordBatchListener.onBatch`	—
3	背压检查	`FileStagingManager.guard`	抛 `BackpressureException`，不 ack，2s 后重试
4	渲染一行	`RecordRenderer.csvRow`	返回 null → 走第 5 步；其余异常即系统级失败
5	脏数据落隔离文件	`RejectStore.save` + `flushAndSync`	抛异常则不 ack，整批重投

#	动作	代码位置	失败后果
1	批量拉取 ≤500 条	`KafkaConsumerConfig`（`max-poll-records`）	—
6	写入分区文件	`StagingWriter.append`	IO 异常 → `IllegalStateException`，不 ack
7	满额封盘	`StagingWriter.seal`	同上
8	移入 pending	`FileStagingManager.moveToPending`	同上（文件仍在 stage，重启可恢复）
9	批末刷盘	`StagingWriter.append` 尾部 `flush + fsync`	同上
10	提交 offset	`ack.acknowledge()`（同步提交）	提交失败由容器重试，数据仍会重投
11	备份	`FileUploadService.backup`	备份失败 → 该文件本轮不上传，留在 pending
12	上传	`CommonsNetFtpUploader.upload`	抛异常 → 计数 +1，下轮重试
13	删 pending	`FileUploadService.deletePending`	删除失败 → 下轮重复上传（远端同名覆盖）

2.5 部署形态

- 单实例进程，内嵌 Tomcat 仅承载运维接口（默认 8080），不承载业务流量。
- 多实例可行：同 `group-id` 下按分区自动分摊；但需注意文件名冲突与 FTP 目录共享问题（见 10.3）。
- 依赖只有两个外部系统：Kafka 集群、FTP 服务器。无数据库、无配置中心、无注册中心。

三、功能清单

3.1 功能全景

编号	功能	功能域	触发方式	关键配置
F-01	配置装载与启动校验	接入	进程启动	全部 yml
F-02	集群节点解析	接入	进程启动	`kafka.cluster.*`
F-03	批量消费	接入	持续轮询	`kafka.consumer.*`
F-04	位移提交与投递语义	接入	每批结束	`max-retries`、`group-id`

编号	功能	功能域	触发方式	关键配置
F-01	配置装载与启动校验	接入	进程启动	全部 yml
F-05	31 列字段映射	管道	每条消息	`DataSchema`（硬编码）
F-06	分片写入与滚动	管道	满 5 万行	`records-per-file`
F-07	封盘	管道	满额 / 空闲 / 手动 / 停机	`idle-flush-ms` 等
F-08	崩溃恢复	管道	启动	`recover-on-start`
F-09	脏数据隔离	管道	消息体不可映射	`reject-dir`
F-10	本地备份	投递	每轮上传前	`backup-dir`、`backup-retention-days`
F-11	FTP 上传与校验	投递	每 60s / 手动	`ftp.*`
F-12	失败重试与隔离	投递	上传异常	`upload-max-attempts`
F-13	备份清理	投递	每天 03:30	`backup-cleanup-cron`
F-14	背压保护	稳定性	积压 / 磁盘低水位	`max-pending-files`、`min-free-disk-mb`
F-15	优雅停机	稳定性	停止信号	`server.shutdown`、`timeout-per-shutdown-phase`
F-16	运维 REST 接口	运维	HTTP	`server.port`
F-17	可观测性（日志/指标/健康）	运维	持续	`logging.*`、`management.*`
F-18	外置配置与分离打包	交付	构建 / 部署	`pom.xml`、`dist.xml`
F-19	自动化测试与压测	交付	`mvn test`	`-Dloadtest.*`

3.2 功能域与角色

功能域	包含	主要受益角色
数据接入	F-01 ~ F-04	实施、运维
文件生产	F-05 ~ F-09	下游数据接入
文件投递	F-10 ~ F-13	下游数据接入、运维
稳定性保障	F-14、F-15	运维
运维与交付	F-16 ~ F-19	运维、开发、实施

四、功能详述

4.1 数据接入域

F-01 配置装载与启动校验

功能说明：启动时把 `config/application.yml` 绑定到 4 个配置类，并对关键项做校验；校验不过直接启动失败，不带病运行。

规则

校验项	规则	不过的后果
主题非空	`kafka.cluster.topic` 必须有值	抛 `IllegalStateException("kafka.cluster.topic 未配置")`
主题格式	只允许 `[a-zA-Z0-9._-]`，长度 1~249	抛 `主题名非法: xxx`
节点非空	解析后至少 1 个 broker	抛 `…未解析出任何 broker 节点`
目录可写	启动即 `createDirectories(stage/pending)`	IO 异常导致启动失败
FTP 配置	不做启动校验（允许先启后修），`ftp.enabled=false` 时只落本地	运行期上传被跳过

配置优先级（高 → 低）：命令行参数 > 操作系统环境变量 > `config/application.yml`。

代码位置：`KafkaLoaderApplication`（`@EnableConfigurationProperties`）、`ClusterDefinitionProvider.load`、`FileStagingManager` 构造器。

验证方式：启动日志出现 `分片文件目录 | stage=… | pending=… | 每文件 50000 行 | 每行 31 列`。

F-02 集群节点解析

功能说明：从 `kafka.cluster.bootstrap-servers` 解析 broker 列表，产出 `host:port` 字符串供 Kafka 客户端使用。这是**唯一**节点来源，不读外部文件、不查库。

支持的两种写法

```
# 写法一：列表（当前采用，便于逐行注释）
bootstrap-servers:
  - 10.100.1.180:9092
  - 10.100.1.181

# 写法二：逗号分隔串
bootstrap-servers: 10.100.1.180:9092,10.100.1.181:9092
```

解析规则

情况	处理
省略端口（如 `10.100.1.181`）	用 `kafka.cluster.default-port`（默认 9092）
`default-port` 超出 1~65535	兜底 9092
同一行写多个（逗号）	逐个拆开解析
重复 `host:port`	去重，保留首次出现顺序
端口非数字 / 超范围	打 WARN 跳过，不影响其他节点
主机名字符非法	打 WARN 跳过
IPv6（`[::1]:9092`）	保留方括号，这是 Kafka 唯一认的写法；支持 `%zone`
`host:`（冒号后为空）	按缺省端口处理，不算错误
全部非法 / 一个都没配	抛异常，启动失败

代码位置： `ConfigKafkaNodeRepository.toNode / parsePort`、`model.ClusterDefinition.getBootstrapServers`（逗号拼接）。

验证方式： 启动日志 `已装载 Kafka 集群配置 | cluster=DISTRIBUTE | topic=… | 节点数=16 | brokers=…`；接口 `GET /api/admin/status` 的 `nodeCount` / `bootstrapServers`。

F-03 批量消费

功能说明： 以批量模式消费单个主题，每批最多 500 条，3 个并发消费者线程。

关键参数与行为

参数	值	行为影响
`batchListener`	true	监听方法收到 `List<ConsumerRecord>`
`max-poll-records`	500	一批上限，也等于一次写入批量

参数	值	行为影响
batchListener	true	监听方法收到 List<ConsumerRecord>
concurrency	3	3 个消费线程；超过分区数则空转
反序列化	StringDeserializer（key/value）	消息体按字符串交给渲染器
isolation.level	read_committed	不读未提交事务消息
fetch-max-bytes	50MB	限制单次 fetch 内存峰值
poll-timeout-ms	3000	无数据时的等待时长
max-poll-interval-ms	300000	两次 poll 最大间隔，须大于一批落盘耗时

空批处理： `records` 为空时直接 ack 返回，不产生文件。

代码位置： `KafkaConsumerConfig.batchListenerContainerFactory`、`KafkaRecordBatchListener.onBatch`。

F-04 位移提交与投递语义

功能说明： 关闭自动提交，改为「数据落盘成功后手动提交」，实现至少一次投递。

规则

1. `enable.auto.commit=false`； `AckMode.MANUAL`； `syncCommits=true`（同步提交，提交结果确定）。
2. ack 时机在 `appendBatch` **返回之后**——即渲染、写盘、fsync 全部完成。
3. 抛 `BackpressureException`、IO 异常、隔离文件写失败 → **一律不 ack**，异常抛给容器。
4. 容器错误处理器：`FixedBackOff(retry-interval-ms=2000, max-retries=-1)` → 默认无限重试，只打 ERROR 日志，不跳过数据。
5. 无已提交 offset 时 `auto-offset-reset=earliest`，从最早开始读。

语义与代价

场景	结果
正常处理	恰好写一次，offset 前移
fsync 后、ack 前崩溃	重启后该批重投 → 文件里出现重复行（至少一次）

场景	结果
正常处理	恰好写一次，offset 前移
停机时未 ack 的在途批	重启后重投 → 可能重复
`max-retries ≥ 0` 且重试用尽	错误处理器 `setAckAfterHandle(true)` → **跳过并 ack**，会丢数据

> 因此 `kafka.consumer.max-retries` 必须保持 `-1`。这是全局唯一的「配置不当会丢数据」的开关。

代码位置： `KafkaConsumerConfig.kafkaErrorHandler`、`KafkaRecordBatchListener`。

4.2 文件管道域

F-05 31 列字段映射

功能说明：把 JSON 消息体按《数据表字段清单》渲染成一行文本，列数恒为 31、顺序固定，与 payload 内键顺序无关。

处理规则

情况	输出
字段存在且为字符串/数字/布尔	原样文本（数字取原始字面量，不做 double 转换，长整型不丢精度）
字段缺失	`\N`
字段值为 `null`	`\N`
字段值是对象或数组	`\N`（并跳过其子树）
同名字段重复出现	取**最后**一个值
清单外的键（含嵌套结构）	忽略，流式跳过，不整树加载
值内含制表符 / 回车 / 换行	替换为空格（否则整行列错位）
空对象 `{}`	输出 31 个 `\N`，是**合法行**
消息体为空 / 纯文本 / 数组 / 截断 JSON	返回 null → 进隔离区（F-09）
每行结尾	无尾随分隔符

为什么不做类型转换：`BIGINT` 走 `double` 会丢精度，`DATETIME` 重新格式化会漂时区。文件里全是文本，类型信息由下游按清单解释。

代码位置：`RecordRenderer.csvRow / scalar / clean`、`DataSchema`（31 字段唯一定义）。

验证方式： `RecordRendererTest` 10 个用例；接口 `GET /api/admin/schema` 返回的列顺序。

F-06 分片写入与滚动

功能说明： 每个 `(topic, partition)` 一个独立写入器，写满 5 万行即滚动出新文件；一批数据可以跨多个文件（写满就切）。

规则

项	行为
写入器粒度	每分区一个，`synchronized`，同分区内串行
临时文件名	`stage/<topic>/p<partition>-<随机>.writing`，不对外可见
缓冲	`BufferedWriter` 128KB
滚动点	写入过程中 `count >= records-per-file` 立即封盘并开新文件
跨批滚动	一批 500 条可被拆到 2 个文件里（前一个写满）
刷盘	每批结束 `flush`，`fsync-per-batch=true` 时再 `fsync`
行格式	UTF-8，`\n` 换行，`\t` 分隔，无表头

代码位置： `StagingWriter.append`（滚动逻辑在第 59~77 行）、`FileStagingManager.write`。

验证方式： `FileStagingManagerTest.shouldRollNewFileEveryConfiguredRecords`；压测中「单文件行数 ≤ 50000」为硬校验项。

F-07 封盘

功能说明： 把未满的临时文件转成正式文件。封盘动作 = `flush + fsync + close + 原子改名 + 移入 pending`。

四种触发

触发	条件	代码入口	日志
写满	达到 `records-per-file`	`StagingWriter.append`	`分片封盘待上传: xxx.csv (N bytes)`
空闲	距最后一次写入 ≥ `idle-flush-ms`（60s），每 30s 检查一次	`flushIdle` ← 定时任务	`空闲封盘 N 个分片（未写满 50000 条）`

触发	条件	代码入口	日志
写满	达到 records-per-file	StagingWriter.append	分片封盘待上传: xxx.csv (N bytes)
手动	POST /api/admin/files/flush	flushAll	手动封盘 N 个分片
停机	容器销毁	destroy() → flushAll	文件管道已关闭，残留分片已封盘

命名：系统配置名称_yyyyMMddHHmmssSSS.csv；同一毫秒撞名追加 -1、-2；分配前同时查内存占位表与磁盘，**绝不覆盖已有文件**。

空文件不产生：count == 0 时封盘直接返回空列表。

两个「原子」点

- 本地：Files.move(…, ATOMIC_MOVE)，不支持时退化为 REPLACE_EXISTING。
- 目录间：stage/<topic>/ → pending/ 同样走 ATOMIC_MOVE，所以 pending 里不会出现半截文件。

代码位置：StagingWriter.seal / sealIfIdle / sealNow、FileStagingManager.moveToPending、FileNameAllocator.allocate。

F-08 崩溃恢复

功能说明：进程异常退出（kill -9、断电、OOM）会留下未封盘的 .writing 文件；启动时自动把它们转成待上传文件，避免数据停在 stage 目录。

规则

情况	处理
recover-on-start=false	跳过，打 WARN，.writing 保持原样
.writing 内容为 0 行	直接删除
.writing 有内容	统计行数 → 分配正式文件名 → 改名 → 移入 pending
单个文件恢复失败	打 ERROR，继续处理下一个（不影响启动）

注意：这些行在上次进程里可能**未提交 offset**，恢复后 Kafka 还会重投同一批，因此恢复文件与重投数据可能重复（同 F-04 的至少一次语义）。

代码位置：`FileStagingManager.recoverInterruptedFiles`（`@PostConstruct`）、`recoverOne`、`countLines`。

验证方式：`FileStagingManagerTest.shouldRecoverLeftoverWritingFiles`；日志 `崩溃恢复完成：N 个未封盘分片已转入待上传队列`。

F-09 脏数据隔离

功能说明：消息体无法映射成 31 列时，不写主文件（否则整行错位污染下游），而是把「来源 + 原始报文」按天写入隔离文件，并且**照常提交 offset**，不阻塞主链路。

触发条件：payload 为 null / 空白 / 非 JSON 对象（纯文本、数组）/ 截断或非法 JSON。

隔离文件

项	值
路径	`<reject-dir>/<系统配置名称>_reject_yyyyMMdd.csv`
格式	UTF-8，`\t` 分隔，**有表头**
列	`topic`、`partition`、`offset`、`record_ts`、`consumed_at`、`payload`
轮转	按自然日一个文件；当天首次写入时若文件不存在则先写表头
落盘	批次结束统一 `flush + fsync`；写失败则整批不 ack
保留 offset 的用途	可用 `kafka-console-consumer --offset` 精确定位重取

统计：累计条数通过 `GET /api/admin/status` 的 `rejectedRecords` 暴露（进程内计数，重启归零；历史量看隔离文件行数）。

代码位置：`RejectStore`（rotateIfNeeded / save / flushAndSync）、`FileStagingManager.appendBatch` 中的 `row == null` 分支。

验证方式：`RecordRendererTest.shouldRejectPayloadThatIsNotJsonObject`、`FileStagingManagerTest.shouldRouteUnmappablePayloadToRejectFile`。

4.3 文件投递域

F-10 本地备份

功能说明：上传**之前**先把文件复制到本地备份目录并 fsync；备份成功才允许上传。任何时刻数据至少两份（pending + backup，或 backup + FTP）。

规则

项	行为
目录	`backup/dt=YYYY-MM-DD/`（按上传当天分目录）
写方式	先写 `xxx.csv.tmp` → `FileChannel.force(true)` → 改名
幂等	目标已存在且大小相同 → 视为已备份，直接返回成功（不重复复制）
失败	记 ERROR，本轮**跳过上传**，文件留在 pending，下轮重试
保留	默认 7 天，由 F-13 清理
指标	`file.chunks.backup` 计数

代码位置：`FileUploadService.backup`。

验证方式：`FileUploadServiceTest.shouldBackupThenUploadThenDeletePending`；日志 `已本地备份: …`。

F-11 FTP 上传与校验

功能说明：每个文件建立一次独立 FTP 连接，上传完成后回读远端大小比对，一致才算成功。

流程

#	步骤	细节
1	连接	`connect-timeout-ms`（10s），检查 `FTPReply.isPositiveCompletion`
2	登录	`username` / `password`（密码建议环境变量注入）
3	传输模式	强制 `BINARY_FILE_TYPE`
4	模式	`passive-mode=true` 时进入被动模式
5	建目录	`remote-dir` 逐级 `CWD`，不存在则 `MKD`
6	上传	先传 `xxx.csv.uploading`
7	改名	`rename(.uploading → 正式名)`；改名失败则删除临时文件并抛错

#	步骤	细节
1	连接	`connect-timeout-ms`（10s），检查 `FTPReply.isPositiveCompletion`
8	校验	`listFiles` 回读远端字节数，必须等于本地大小（`verify-size=true`）
9	收尾	`logout` + `disconnect`（异常吞掉，不影响结果）
10	清理	成功后删除本地 pending（`delete-after-upload=true`）

失败处理：任何一步抛 `IOException` → F-12 计数重试；文件保留在 pending，本地备份已在 F-10 完成。

编码：控制连接编码 `ftp.encoding`（默认 UTF-8），文件名含中文时须与服务端一致，否则远端乱码。

代码位置：`CommonsNetFtpUploader.upload / ensureRemoteDir / remoteSize`。

验证方式：`CommonsNetFtpUploaderTest`（MockFtpServer，真实 FTP 协议）；压测场景四验证中断 8s 后自动补齐。

F-12 失败重试与隔离

功能说明：上传失败的文件保留在 pending，按轮次重试；超过上限移入 `failed/` 并写原因文件，不静默丢弃。

规则

项	行为
重试节奏	由定时任务驱动，默认每 60s 一轮；手动接口也可触发
计数	进程内 `Map<文件名, 次数>`，**重启归零**
上限	`upload-max-attempts`（默认 10）
达上限	`Files.move` 到 `failed/`，同时写 `xxx.csv.reason`（纯文本原因）
未达上限	打 WARN「上传失败(3/10): xxx」，并附失败原因，继续留在 pending
互斥	与定时任务共用 `ReentrantLock`，并发调用返回 `executed=false`

注意：一批文件里某个失败不阻断后续文件（循环内捕获异常继续）。

代码位置：`FileUploadService.handleFailure / moveToFailed`。

验证方式：`FileUploadServiceTest.shouldKeepPendingWhenUploadFails`、`shouldMoveToFailedAfterMaxAttempts`。

F-13 备份清理

功能说明：按保留天数删除过期备份目录，只删本地备份，不影响 pending / FTP。

规则

项	行为
触发	Quartz cron `0 30 3 * * ?`（每天 03:30）
判定	目录名 `dt=yyyy-MM-dd` 解析出的日期 < 今天 − `backup-retention-days` 则删除
下限	保留天数最小按 1 处理
删除	递归删目录内文件，返回删除文件数
容错	非 `dt=` 前缀目录跳过；解析异常记 ERROR 不中断

代码位置：`FileUploadService.cleanupBackup / deleteRecursively`。

验证方式：`FileUploadServiceTest.shouldCleanupExpiredBackup`；日志 `已清理 N 个过期备份文件（保留 7 天）`。

4.4 稳定性保障域

F-14 背压保护

功能说明：当上传追不上写入、或磁盘将满时，主动停止消费且不提交 offset，让数据留在 Kafka，避免把节点磁盘写满拖垮机器。

两个触发条件（任一命中即触发）

条件	默认阈值	含义
待上传文件数	`pending ≥ 2000`	FTP 侧吞吐不足
磁盘剩余	`< 2048MB`	本地空间告急（按 stage 目录所在盘）

行为

1. `appendBatch` 入口 `guard()` 检查，抛 `BackpressureException`。
2. 监听器捕获后**重新抛出**，不 ack → 容器按 `retry-interval-ms=2000` 重试同一批。
3. offset 不前进，Kafka 侧自然降速（消费位点卡住）。

4. 条件恢复（pending 被上传清空 / 磁盘释放）后自动继续，无需重启。

时效性：pending 计数有 1 秒缓存（`pendingCount()` 每 1000ms 最多扫一次盘），因此最多 1 秒延迟；粒度是「批」，单批 ≤500 条可能越过阈值。

代码位置：`FileStagingManager.guard / pendingCount`、`KafkaRecordBatchListener` 的 catch 分支。

验证方式：`FileStagingManagerTest.shouldRejectWhenPendingBacklogExceeded`；日志 `背压触发，暂停消费本批 500 条：…`。

F-15 优雅停机

功能说明：收到停止信号后先停止拉新消息，把在途批次处理完、残留分片封盘，再退出。

规则

项	值
`server.shutdown`	`graceful`
`timeout-per-shutdown-phase`	60s（超过则强制退出，未提交 offset 由 Kafka 重投）
销毁钩子	`FileStagingManager.destroy()`：`flushAll` → 关闭各写入器 → 关闭隔离文件
关闭隔离文件	关闭前 `flush + fsync`，不留缓冲数据

注意：`destroy()` 只封盘，不触发上传；残留文件在下次启动时由 F-08/正常扫描处理（pending 里的文件下轮上传）。

代码位置：`FileStagingManager.destroy`（`DisposableBean`）。

4.5 运维与交付域

F-16 运维 REST 接口

功能说明：提供 5 个接口用于手动推进和状态查询，Swagger 页面可视化调试。详见第七章。

共同特征：无鉴权、只读或幂等触发、不修改配置、不改变消费位点。

F-17 可观测性（日志 / 指标 / 健康）

功能说明：三层可观测手段。

层	内容
日志	异步滚动日志，按天 + 200MB 切分，gzip 归档，保留 15 天 / 总量 20GB；业务包 DEBUG
指标	Micrometer 计数器 `file.chunks.backup` / `file.chunks.uploaded` / `file.chunks.upload_failed`
健康	`/actuator/health`（存活探测）、`/actuator/info`、`/actuator/metrics`
状态快照	`GET /api/admin/status` 汇总集群、FTP、管道、隔离计数

日志架构：logback 由外置 `config/logback-spring.xml` 提供（`-Dlogging.config` 加载，不进 jar）；异步队列 8192，队列满时阻塞业务线程而不丢日志；进程早期输出单独写 `logs/console.out`，避免抢文件句柄。

代码位置：`src/main/resources/logback-spring.xml`、`config/application.yml` 的 `logging.*` / `management.*`。

F-18 外置配置与分离打包

功能说明：业务 jar 与配置、依赖彻底分离，改配置不用重新打包。

产物	内容	大小
`kafka-file-loader.jar`	仅业务 class（`layout=ZIP` + `includes` 排除依赖）	约 228KB
`lib/`	63 个第三方 jar	约 41MB
`config/application.yml`	全部业务配置	—
`config/logback-spring.xml`	日志策略	—
`bin/start.sh`、`bin/start.bat`	启停脚本	—
`kafka-file-loader-1.0.0.tar.gz`	上述内容打包（无顶层目录，解压即用）	约 39.5MB

三项配套构建配置：`spring-boot-maven-plugin` 的 `layout=ZIP` + `includes`（启用 `PropertiesLauncher`）、`maven-jar-plugin` 的 `Loader-Path=lib`、`resources` 排除 `application.yml` / `logback-spring.xml`。

启动命令（脚本最终执行）：

```
java -Dfile.encoding=UTF-8 \
     -Dloader.path=lib \
     -Dlogging.config=config/logback-spring.xml \
     -jar kafka-file-loader.jar \
     --logging.file.path=logs
```

代码位置：`pom.xml`、`src/assembly/dist.xml`、`bin/start.sh`、`bin/start.bat`。

F-19 自动化测试与压测

功能说明：50 个功能用例 + 6 个压测用例，覆盖关键规则；压测链路不绕过任何一环。

测试类	用例数	覆盖
`RecordRendererTest`	10	31 列顺序、乱序键、缺字段、嵌套、重复键、制表符、数字精度、空对象、非对象拒绝
`ConfigKafkaNodeRepositoryTest`	11	列表/逗号写法、默认端口、去重、IPv6、非法跳过、全非法失败、端口兜底
`ClusterDefinitionProviderTest`	8	主题校验、快照不可变、初始化前访问、异常透传
`FileStagingManagerTest`	10	滚动、空闲封盘、崩溃恢复、背压、停机封盘、列序、隔离、文件名唯一/回退/清洗
`FileUploadServiceTest`	5	备份→上传→删除、失败保留、超限隔离、FTP 关闭跳过、过期清理
`CommonsNetFtpUploaderTest`	2	真实 FTP 协议上传字节一致、服务不可达快速失败
`RefreshingConsumerFactoryTest`	4	每次建连取最新地址、空值忽略、反序列化器实例、groupId 覆盖
`PipelineLoadTest`	6	四场景压测 + 环境信息 + 报告生成

压测硬校验（不只看吞吐）：零丢失（备份总行数 = 生产条数）、每行恰好 31 列、单文件 ≤5 万行、FTP 回读字节一致、`rejectedRecords=0`、`failed/` 与 pending 无残留。

五、文件与数据规格

5.1 文件命名

```
DISTRIBUTE_20261022173000123.csv
   |          |       └── 固定扩展名（CSV 格式，\t 分隔）
   |          └────────── 封盘时刻，17 位 yyyyMMddHHmmssSSS（可按名排序）
   └──────────────────────── 系统配置名称（file.system-name，未配置回退 kafka.cluster.name）
```

- 同一毫秒多个分区封盘：`DISTRIBUTE_20261022173000123-1.csv`、`-2`……
- 名字清洗：只保留字母、数字、中文、`.`、`_`、`-`，其余转 `_`；空值回退 `kafka`。
- pending / backup / FTP 全链路同名，便于端到端核对。

5.2 行格式

项	值
编码	UTF-8（无 BOM）
分隔符	制表符 `\t`
换行	`\n`
表头	无
列数	固定 31
缺失值	`\N`（反斜杠 + N）
行尾	无尾随分隔符

> 用 Excel 打开需在导入向导里把分隔符选为「制表符」。下游按位置装载（LOAD DATA / sqoop / DataX / 自写脚本）可直接按 `\t` 切分。

5.3 31 字段字典

列顺序 = 清单序号 1~31，**不可调整**（下游按位置装载，顺序错了不报错，只会静默串列）。

序	字段名	类型	含义
1	`md_id`	STRING	表主键
2	`clue_id`	STRING	线索 ID/日志查询标识

序	字段名	类型	含义
1	md_id	STRING	表主键
3	clue_src_sys	STRING	线索布控/日志查询源
4	clue_dst_sys	STRING	线索布控目的地址/日志产生源
5	capture_time	BIGINT	截获时间(上线时间)
6	protocol	STRING	协议代码(本数据)
7	original_protocol	STRING	原始数据协议
8	data_source3	STRING	数据来源系统分类代码
9	mwbs	STRING	记录唯一标识
10	sensitive_level	STRING	数据敏感级别
11	saved_time	BIGINT	信息入库时间
12	remark	STRING	数据来源系统备注
13	data_check_id	STRING	对账标识 ID
14	bunit	STRING	采集地单位名称
15	data_set_name	STRING	科信数据集名称
16	dzwjwz	STRING	电子文件位置
17	asjxrybh	STRING	案事件相关人员编号
18	asjbh	STRING	案事件编号
19	zjhm	STRING	常用证件 证件号码
20	xm	STRING	姓名
21	xbdm	STRING	性别
22	xbmc	STRING	性别名称
23	mzdm	STRING	民族
24	mzmc	STRING	民族名称
25	csrq	DATETIME	出生日期 日期值下限

序	字段名	类型	含义
1	md_id	STRING	表主键
26	xzzqhdm	STRING	户籍地址 行政区划代码
27	xzzqhmc	STRING	户籍地址 地址名称
28	criminal_time	DATETIME	抓获日期
29	gajgjgdm	STRING	抓获单位 公安机关机构代码
30	gajgjmc	STRING	抓获单位 公安机关名称
31	synchro_time	DATETIME	更新时间

类型统计：STRING 26 个、BIGINT 2 个、DATETIME 3 个。类型仅用于下游建表对照，文件内一律文本。

5.4 取值规则速查

输入	输出
{"xm":"张三"}	第 20 列为 张三，其余 30 列为 \N
{"capture_time":1698000000123}	1698000000123（不转 double）
{"xm":null}	\N
{"xm":{"a":1}}	\N
{"xm":"张\t三"}	张 三（制表符被替换）
{"extra":"x"}	extra 被忽略，全部 \N
{}	31 个 \N（合法行）
[1,2] / abc / 空串	不进主文件，进隔离区

5.5 隔离文件格式

项	值
路径	data/file-reject/<系统配置名称>_reject_yyyyMMdd.csv
表头	topic、partition、offset、record_ts、consumed_at、payload（\t 分隔）
分隔符	\t

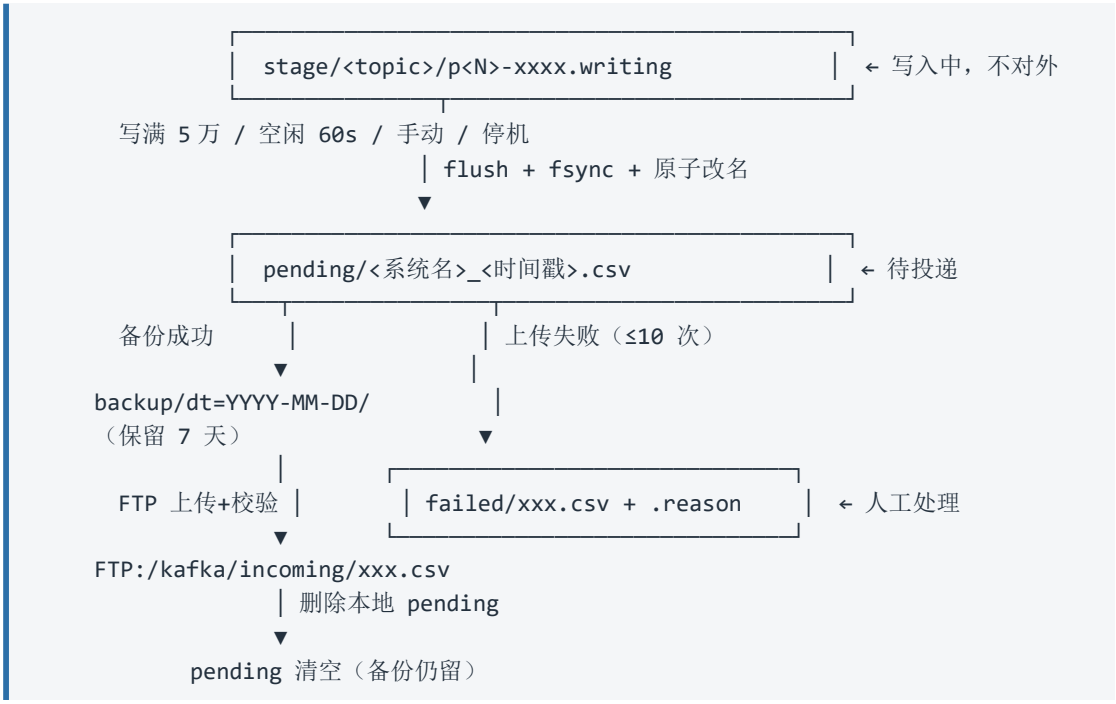
项	值
路径	`data/file-reject/<系统配置名称>_reject_yyyyMMdd.csv`
payload	原始报文，其中制表符/换行替换为空格（JSON 其余内容保留）
用途	事后补数与排查；用 offset 可精确重取

5.6 编码与时区

- 文件与隔离文件统一 UTF-8；FTP 控制连接编码独立可配（`ftp.encoding`）。
- 时间类字段**不做任何转换**，原样输出生产方的文本，避免时区漂移。
- 文件名时间戳取服务器本地时区（`LocalDateTime.now()`），备份目录日期同理。

六、文件状态机与目录

6.1 状态流转



6.2 五个目录

目录	配置项	内容	谁清理
`data/file-stage/<topic>/`	`file.stage-dir`	`*.writing` 写入中文件	封盘时移走；空文件重启时删

目录	配置项	内容	谁清理
data/file-stage/<topic>/	file.stage-dir	*.writing 写入中文件	封盘时移走；空文件重启时删
data/file-pending/	file.pending-dir	已封盘待投递	上传成功后删除
data/file-backup/dt=日期/	file.backup-dir	本地备份	每天 03:30 按保留天数清理
data/file-failed/	file.failed-dir	重试超限 + .reason	人工
data/file-reject/	file.reject-dir	脏数据按天归档	人工（按需归档）

前三个目录启动时自动创建；failed / reject 在首次使用时创建。

6.3 各环节失败的处理对照

失败点	数据在哪	是否 ack	恢复方式
渲染异常（非脏数据类）	Kafka	否	2s 后重投
隔离文件写失败	Kafka	否	重投
分片写盘失败（磁盘满/IO）	Kafka	否	修复后重投；或背压兜底
封盘/移入 pending 失败	stage 残留	否	重启后 F-08 恢复
备份失败	pending	不涉及	下轮重试备份
FTP 上传失败	pending	不涉及	下轮重试，10 次后进 failed
删除 pending 失败	pending	不涉及	下轮重传（远端同名覆盖）

七、接口规格

7.1 接口总览

方法	路径	作用	幂等
POST	/api/admin/files/flush	立即封盘所有写入器	是（无数据返回 0）
POST	/api/admin/files/upload	立即备份并上传 pending	是（并发时跳过）
GET	/api/admin/files/stats	管道文件统计	是
GET	/api/admin/schema	31 字段定义	是

方法	路径	作用	幂等
POST	`/api/admin/files/flush`	立即封盘所有写入器	是（无数据返回 0）
GET	`/api/admin/status`	服务状态总览	是

- Swagger UI：`http://<host>:8080/swagger-ui.html`；OpenAPI JSON：`/v3/api-docs`。
- 返回 JSON 不输出 null 字段（`spring.jackson.default-property-inclusion=non_null`）。
- 无鉴权；建议只在内网/堡垒机可达。

7.2 POST /api/admin/files/flush

把未写满的分片立刻封盘（低峰期、停机前手动推进）。

```
{
  "sealedFiles": 3,
  "pendingFiles": 3,
  "message": "已封盘 3 个分片"
}
```

字段	说明
`sealedFiles`	本次封盘数量；无数据时为 0，`message` 为「当前没有待封盘数据」
`pendingFiles`	封盘后 pending 目录文件数（实时统计）

7.3 POST /api/admin/files/upload

立即执行一轮「备份 → 上传 → 校验 → 删除」。与定时任务共用锁，不会并发。

```
{
  "executed": true,
  "scanned": 3,
  "uploaded": 3,
  "failed": 0,
  "backupSkipped": 0,
  "elapsedMs": 1240,
  "message": "上传完成"
}
```

字段	说明

`executed`	是否真的执行；`false` 表示 FTP 关闭或已有任务在跑
`scanned`	扫描到的 pending 文件数
`uploaded` / `failed`	成功 / 失败个数
`backupSkipped`	因备份失败而跳过上传的个数
`elapsedMs`	本轮耗时
`message`	`上传完成` / `没有待上传文件` / `FTP 未启用，跳过上传` / `已有上传任务在执行，本次跳过`

7.4 GET /api/admin/schema

返回列顺序，供下游建表与装载脚本核对。

```
{
  "fieldCount": 31,
  "delimiter": "\\t",
  "fileExtension": ".csv",
  "fields": [
    { "index": 1, "name": "md_id", "type": "STRING", "description": "表主键" },
    { "index": 2, "name": "clue_id", "type": "STRING", "description": "线索 ID/日志查询标识" }
  ]
}
```

> `delimiter` 是字面量 `\t`（两个字符），不是真实制表符。

7.5 GET /api/admin/files/stats

```
{
  "stageFiles": 3,
  "pendingFiles": 0,
  "pendingBytes": 0,
  "backupFiles": 24,
  "backupBytes": 10485760,
  "failedFiles": 0,
  "diskFreeMb": 51200,
  "recordsPerFile": 50000,
  "stageDir": "/app/data/file-stage",
  "pendingDir": "/app/data/file-pending",
  "backupDir": "/app/data/file-backup"
}
```

字段	说明
`stageFiles`	stage 目录下文件数（**递归**，含各 topic 子目录）

字段	说明
`stageFiles`	stage 目录下文件数（**递归**，含各 topic 子目录）
`pendingFiles` / `pendingBytes`	待投递文件数与字节数
`backupFiles` / `backupBytes`	备份文件数与字节数（递归含 dt 子目录）
`failedFiles`	失败区文件数（含 `.reason` 文件，故实际是「文件数」而非「待处理数」×2）
`diskFreeMb`	stage 所在磁盘剩余 MB

7.6 GET /api/admin/status


```
{
  "time": "2026-10-26T09:12:33.421",
  "clusterName": "DISTRIBUTE",
  "topic": "DISTRIBUTE_SQAJXYRXX_TO",
  "systemName": "DISTRIBUTE",
  "fileNamePattern": "DISTRIBUTE_<yyyyMMddHHmmssSSS>.csv",
  "bootstrapServers": "10.100.1.180:9092,10.100.1.181:9092",
  "nodeCount": 16,
  "nodeSource": "application.yml (kafka.cluster.bootstrap-servers)",
  "ftpEnabled": true,
  "ftpHost": "127.0.0.1:21",
  "ftpRemoteDir": "/kafka/incoming",
  "uploadRunning": false,
  "fieldCount": 31,
  "rejectedRecords": 0,
  "rejectDir": "/app/data/file-reject",
  "files": {
    "stageFiles": 3,
    "pendingFiles": 0,
    "pendingBytes": 0,
    "backupFiles": 24,
    "backupBytes": 10485760,
    "failedFiles": 0,
    "diskFreeMb": 51200,
    "recordsPerFile": 50000,
    "stageDir": "/app/data/file-stage",
    "pendingDir": "/app/data/file-pending",
    "backupDir": "/app/data/file-backup"
  }
}
```


- `nodeSource` 固定为配置文件，不存在「从数据库/配置中心加载」的情况。
- `uploadRunning` 反映上传锁状态，可用于判断是否卡在上传。
- `rejectedRecords` 为进程内累计，重启归零。

7.7 Actuator

路径	用途
`/actuator/health`	存活探测（K8s liveness/readiness）
`/actuator/info`	应用信息
`/actuator/metrics`	指标；重点 `file.chunks.uploaded`、`file.chunks.upload_failed`、`file.chunks.backup`

仅暴露 `health,info,metrics`，其余端点不对外。

八、非功能特性

8.1 性能实测

环境：Windows 11 / 18 核 / JDK 1.8.0_503 / 最大堆 7195MB / 单条报文 957B（落盘后每行 416B）。

场景	条数	端到端吞吐	落盘吞吐	文件数	丢数据
生产默认（fsync=true，5 万行/文件）	200,000	42,061 条/秒	16.71 MB/秒	6	0
持续压力	500,000	55,109 条/秒	21.90 MB/秒	12	0
对照（fsync=false）	200,000	40,453 条/秒	16.07 MB/秒	6	0
FTP 中断 8s 后恢复	100,000	中断期暂停	恢复后 967ms 补齐	3	0

结论：

1. 单实例稳定处理数万条/秒，吞吐随数据量不衰减（50 万条比 20 万条高约 30%，是固定开销被摊薄）。
2. **瓶颈不在 fsync**：500 条/批每批只刷一次，SSD 上开销被 JSON 渲染与写盘掩盖。
3. 真正上限更可能来自 FTP 带宽——因此设计了背压（F-14）而不是无限堆积。

8.2 可靠性设计矩阵

风险	机制	效果

断电/崩溃丢已 ack 数据	每批 `flush + fsync` 后才 ack	已 ack 数据一定在磁盘
下游读到半截文件	本地 `.writing` + 原子改名；远端 `.uploading` + rename	正式名文件永远完整
上传了半截却当成功	回读远端字节数比对	不完整不算成功
备份失败导致只有一份	备份失败则不上传	任何时刻至少两份
FTP 长时间不可用	pending 保留 + 按轮重试	网络恢复后自动补齐
永久失败文件被静默丢弃	超限移入 failed 并写原因	可人工介入
脏数据污染主文件	隔离到 reject，主文件恒 31 列	下游不会整行错位
磁盘被写满	积压/水位背压，停止消费不 ack	保护节点
停机产生半截分片	graceful shutdown + 销毁钩子封盘	重启可恢复
配置写错连不上	启动校验主题/节点，非法即失败	不带病运行

8.3 并发模型

组件	并发度	同步手段
Kafka 消费	`concurrency=3` 线程	每线程处理不同分区
分片写入	每分区一个 writer	`synchronized`（同分区串行）
文件名分配	全局	`synchronized` + 内存占位 + 磁盘检查
上传	单线程	`ReentrantLock.tryLock()`
定时任务	默认调度线程池单线程	三个任务串行（见 10.7）
隔离文件写入	多消费线程可能并发	`synchronized`（RejectStore 方法级）

8.4 资源占用

项	估算
内存	堆 1~2GB 足够（单批 ≤500 条 × 报文大小 + 128KB 写缓冲/分区）
句柄	每分区 1 个文件句柄 + 上传时 1 个 FTP 连接
磁盘	pending 峰值 ≈ 2000 文件 × 5 万行 × 416B ≈ 41GB（触发背压前的理论上限）；实际按吞吐与 FTP 带宽估算
网络	上传占用 = 落盘吞吐（约 16~22MB/s 量级）

8.5 安全

项	现状	建议
运维接口	无认证	只在内网开放，或前置网关鉴权
FTP 密码	支持 `${FTP_PASSWORD:默认值}` 环境变量注入	生产用环境变量，不写明文
数据内容	明文落盘与传输（FTP 非加密）	如需加密改用 SFTP/FTPS（替换 `FtpUploader` 实现即可）
日志	不打印消息体全文（只打条数与文件名）	保持该约定，避免敏感数据入日志

九、配置项 → 功能映射

9.1 影响行为的核心配置

配置	默认	影响的功能	改动风险
`kafka.cluster.topic`	`DISTRIBUTE_SQAJXYRXX_TO`	F-02、F-03	改错则无消息，且不报错
`kafka.cluster.bootstrap-servers`	16 个节点	F-02	改错启动失败
`kafka.consumer.group-id`	`kafka-hive-loader`	F-04	改它 = 换新消费者，earliest 会重投历史
`kafka.consumer.concurrency`	3	F-03	超过分区数空转
`kafka.consumer.max-poll-records`	500	F-03、F-06	调大提升吞吐，故障重投范围也变大
`kafka.consumer.max-retries`	-1	F-04	**改成 ≥0 会丢数据**
`file.records-per-file`	50000	F-06	影响单文件大小与下游装载批次
`file.fsync-per-batch`	true	F-06、F-01	关掉违背稳定性前提
`file.idle-flush-ms`	60000	F-07	调大低峰期数据滞留更久
`file.flush-check-interval-ms`	30000	F-07	检查粒度
`file.upload-interval-ms`	60000	F-11	首轮上传也延迟该时长
`file.max-pending-files`	2000	F-14	调小更早背压
`file.min-free-disk-mb`	2048	F-14	调小磁盘风险更高
`file.upload-max-attempts`	10	F-12	调大容忍更久，但积压更久
`file.backup-retention-days`	7	F-13	备份保留窗口

配置	默认	影响的功能	改动风险
kafka.cluster.topic	DISTRIBUTE_SQAJXYRXX_TO	F-02、F-03	改错则无消息，且不报错
file.system-name	DISTRIBUTE	F-05、F-07	影响文件名前缀（下游可能按名筛选）
file.recover-on-start	true	F-08	关掉会留下 stage 残留
ftp.enabled	true	F-11	关闭后只落 pending，不出网
ftp.verify-size	true	F-11	关掉等于放弃完整性校验
ftp.delete-after-upload	true	F-11	关掉 pending 会持续堆积直至背压
server.port	8080	F-16	仅运维接口

9.2 配置改动生效方式

配置类别	生效方式
业务配置（kafka/file/ftp）	改 config/application.yml 后**重启**
日志级别	可改 yml 后重启，或启动参数 --logging.level.xxx=DEBUG
日志滚动参数	JVM 参数 -DLOG_MAX_FILE_SIZE / -DLOG_MAX_HISTORY
端口	--server.port=18090 或环境变量 SERVER_PORT

十、边界条件与已知限制

10.1 数据语义

#	限制	影响	应对
1	至少一次语义，无去重	崩溃/重投会产生重复行	下游按 mwbs（记录唯一标识）或主键去重
2	字段名精确匹配、区分大小写	XM 与 xm 不匹配 → \N	生产侧保证键名与清单一致
3	\N 无转义	字符串本身是 \N 时无法与缺失区分	生产侧避免该字面量，或下游按业务规则处理
4	不做长度/类型校验	超长值、非法日期原样落盘	下游装载时校验；必要时在渲染层加规则
5	制表符/换行被替换为空格	payload 内该类字符不保留原样	已保证不串列；如需原样改分隔符
6	只消费一个主题	多主题需多实例	复制配置、改 topic 与 system-name

#	限制	影响	应对
1	至少一次语义，无去重	崩溃/重投会产生重复行	下游按 `mwbs`（记录唯一标识）或主键去重
7	字段清单硬编码在 `DataSchema`	改列需改代码 + 重新打包 + 同步下游	变更走版本发布，勿只改下游

10.2 运维接口与进程

#	限制	影响	应对
8	接口无鉴权	任何可达者都能触发封盘/上传	网络隔离；不要暴露公网
9	重试计数在内存	重启后重试次数归零	长期失败文件可能反复重试，需看日志/监控
10	`rejectedRecords` 为进程内计数	重启归零	历史量统计隔离文件行数
11	配置不支持热更新	改节点/主题需重启	计划内变更窗口

10.3 多实例部署

#	限制	影响	应对
12	多实例同一 `system-name` 指向同一 FTP 目录	同一毫秒封盘理论上可能同名（概率极低，`allocate` 只查本地磁盘）	每实例 `file.system-name` 加实例后缀
13	多实例不得共享本地数据目录	pending/backup 会互相干扰	每实例独立工作目录
14	同一消费组实例数 > 分区数	多余实例空转	按分区数规划实例数

10.4 调度与时效

#	限制	影响	应对
15	三个定时任务共用默认单线程调度器	一次长上传会推迟空闲封盘与备份清理	需要时调大 `spring.task.scheduling.pool.size`（当前未配）
16	pending 计数缓存 1 秒	背压判断最多延迟 1 秒	可接受；精确判断用 `/files/stats`
17	背压按批生效	单批最多 500 条越过阈值	阈值留余量（默认 2000 文件 / 2GB）

#	限制	影响	应对
15	三个定时任务共用默认单线程调度器	一次长上传会推迟空闲封盘与备份清理	需要时调大 `spring.task.scheduling.pool.size`（当前未配）
18	首轮上传延迟一个 `upload-interval-ms`	启动后 60s 才第一次自动上传	需要立即投递时手动调 `/files/upload`

10.5 投递与外部依赖

#	限制	影响	应对
19	FTP 明文传输	无加密	换 SFTP/FTPS 实现（`FtpUploader` 接口已抽象）
20	`verify-size` 依赖服务端 `LIST`	个别服务端返回异常会导致校验失败（remote=-1）	必要时关 `verify-size`，但失去完整性校验
21	远端同名文件存在时 `rename` 可能失败（部分服务端）	该文件持续失败直至进 `failed`	定期清理远端同名文件；或确认服务端允许覆盖
22	上传串行、单连接	大文件或高延迟链路吞吐受限	提高 FTP 带宽；文件级别已并行不了（简化优先）
23	备份跨天重试会在新日期目录再备份一份	备份目录出现同名双份	占用可忽略；清理按目录日期执行

十一、测试与验收

11.1 测试执行

```
mvn test                                    # 全部用例
mvn test -Dtest=RecordRendererTest          # 单类
mvn test -Dtest=PipelineLoadTest            # 压测（约 80s，默认 80 万条量级）
mvn test -Dtest=PipelineLoadTest -Dloadtest.records=1000000
```

压测报告自动写入 `target/loadtest/report.md`；历史结论见 `docs/loadtest.md`。

11.2 功能验收清单

#	验收项	通过标准
1	启动即校验	删掉 topic 或把节点写错，启动直接失败并给出明确原因
2	节点解析	`/status` 的 `nodeCount` 等于配置中有效节点数；重复/非法项被跳过

#	验收项	通过标准
1	启动即校验	删掉 topic 或把节点写错，启动直接失败并给出明确原因
3	分片滚动	单文件行数 ≤ 50000，文件名形如 `DISTRIBUTE_2026...csv`
4	列数正确	随机抽一行按 `\t` 切开恰好 31 列
5	缺字段占位	只含 `xm` 的消息，其余 30 列为 `\N`
6	脏数据隔离	发一条纯文本，主文件无该行，`file-reject/` 出现带 offset 的记录，且 offset 正常前移
7	空闲封盘	停止生产 60~90s 后，未满分片自动进 pending
8	崩溃恢复	`kill -9` 后重启，stage 里的 `.writing` 变成 pending 文件且行数不变
9	备份先于上传	pending 出现后，`backup/dt=今天/` 有同名同大小文件
10	远端无半截	FTP 目录中不存在 `.uploading` 残留；文件大小与本地一致
11	上传失败不丢	停 FTP 后 pending 持续保留，重启 FTP 后自动补齐
12	超限隔离	把 `upload-max-attempts=1` 并保持 FTP 不可用，文件进 `failed/` 且有 `.reason`
13	背压生效	把 `max-pending-files=1`，日志出现「背压触发」，消费位点停止前进
14	优雅停机	`stop` 后无 `.writing` 残留（或残留为空），日志有「文件管道已关闭」
15	接口可用	5 个接口返回结构与本文一致；并发调用 upload 有一个返回 `executed=false`
16	配置外置	改 `config/application.yml` 的 `file.system-name` 重启后，`/status` 的 `systemName` 与文件名前缀同步变化，jar 未重新打包

十二、代码结构地图

```
src/main/java/com/example/kafkaloader/
├── KafkaLoaderApplication.java        启动类（配置绑定 + 开启定时任务）
├── api/
│   └── AdminController.java           5 个运维接口 + Swagger 注解
├── cluster/
│   └── ClusterDefinitionProvider.java 启动校验、不可变集群快照
├── config/
│   ├── ClusterConfig.java             节点仓储与 provider 的装配
│   ├── KafkaConsumerConfig.java       消费者工厂、批量容器、错误处理器
│   ├── KafkaClusterProperties.java    kafka.cluster.*
│   ├── KafkaConsumerProperties.java   kafka.consumer.*
│   ├── FilePipelineProperties.java    file.*
│   ├── FtpProperties.java             ftp.*
│   └── OpenApiConfig.java             OpenAPI 元信息
├── file/
│   ├── DataSchema.java                31 字段唯一定义（顺序即列序）
│   ├── RecordRenderer.java            JSON → 一行 31 列
│   ├── StagingWriter.java             单分区写入、滚动、封盘
│   ├── FileNameAllocator.java         文件名生成与去重
│   ├── FileStagingManager.java        管道总控：路由、封盘、恢复、背压、统计
│   ├── RejectStore.java               脏数据按天隔离
│   ├── FileUploadService.java         备份 → 上传 → 删除；重试与隔离；备份清理
│   ├── FilePipelineScheduler.java     三个定时任务
│   ├── FileStats.java                 统计 DTO
│   ├── UploadResult.java              上传结果 DTO
│   └── BackpressureException.java     背压信号
├── ftp/
│   ├── FtpUploader.java               上传抽象（可换 SFTP/对象存储）
│   └── CommonsNetFtpUploader.java     commons-net 实现
├── listener/
│   └── KafkaRecordBatchListener.java  批量消费入口，控制 ack 时机
├── model/
│   ├── ClusterDefinition.java         节点 + 主题快照
│   ├── KafkaNode.java                 单节点
│   └── KafkaMessageRecord.java        消息中间模型
├── repository/
│   ├── KafkaNodeRepository.java       节点来源接口
│   └── ConfigKafkaNodeRepository.java yml 解析实现
└── support/
    └── RefreshingConsumerFactory.java 每次建连取最新 broker 地址
```

二次开发指引

需求	改哪里	注意
加/改/删字段	`DataSchema` 的 `FIELDS`	同步改 `RecordRendererTest` 的列数断言与下游建表
换分隔符	`RecordRenderer.DELIMITER`	文件名仍为 `.csv`，需同步下游

需求	改哪里	注意
加/改/删字段	`DataSchema` 的 `FIELDS`	同步改 `RecordRendererTest` 的列数断言与下游建表
投递到 SFTP / 对象存储	实现 `FtpUploader`，替换 `@Service` 实现	`FileUploadService` 无需改动
多主题	复制 `@KafkaListener` 或改用 topic 列表	文件名前缀需区分，避免 FTP 混淆
增加运维接口	`AdminController`	保持无状态、幂等
调吞吐	`concurrency` / `max-poll-records` / `records-per-file`	先确认分区数与 FTP 带宽

附录 A 日志关键字

阶段	关键字	级别
管道就绪	分片文件目录	INFO
集群装载	已装载 Kafka 集群配置	INFO
消费初始化	Kafka 消费初始化	INFO
封盘	分片封盘待上传:	INFO
空闲封盘	空闲封盘	INFO
手动封盘	手动封盘	INFO
崩溃恢复	崩溃恢复完成 / 恢复未封盘分片	WARN
脏数据	无法映射到字段清单 / 隔离文件就绪	WARN / INFO
备份	已本地备份:	INFO
上传成功	FTP 上传成功:	INFO
上传失败	上传失败(1/10)	WARN
超限隔离	文件上传连续失败已达上限	ERROR
上传汇总	文件上传完成（含扫描/成功/失败/耗时）	INFO
背压	背压触发，暂停消费本批	WARN
清理	已清理 N 个过期备份文件	INFO
停机	文件管道已关闭	INFO

附录 B 常见问题

现象	定位	处置
启动失败「未解析出任何 broker 节点」	节点格式非法或为空	按 1.2 的两种写法核对；看 WARN 里被跳过的具体地址

现象	定位	处置
启动失败「未解析出任何 broker 节点」	节点格式非法或为空	按 1.2 的两种写法核对；看 WARN 里被跳过的具体地址
启动正常但一直无文件	topic 写错 / 上游无数据 / 积压不足 5 万且未到空闲时间	`/status` 看 topic；等 60s 空闲封盘；手动 `/files/flush`
pending 一直涨	FTP 不通或带宽不足	看「上传失败(x/10)」原因；恢复 FTP 后自动追平；接近 2000 会背压
文件里某列整体偏移	字段清单被改而下游未同步	用 `/schema` 核对；回滚清单或同步下游
行数比预期多	重投导致的重复（至少一次）	下游去重；不要用行数做精确对账
`failed/` 出现文件	连续 10 次上传失败	看 `.reason`；修好 FTP 后手工移回 pending
隔离文件持续增长	生产侧报文格式变了（非 JSON 对象）	抽样看 payload；联系上游；必要时回灌
远端出现 `.uploading` 残留	上传中途断连	手工删除；本服务下轮会重传正式名
端口不是 8080	环境变量 `SERVER_PORT` 覆盖	执行 `env` 检查是否有 `SERVER_PORT`